



Selenium

Python

PyTest

Behave



SELENIUM PYTHON AUTOMATION

MODULE-WISE LAB WORKBOOK

Modules 1, 2 & 3 — Locators • Page Object Model • PyTest • Behave BDD

3

MODULES

12

EXPERIMENTS

1

GITHUB REPOSITORY

MODULE 1

Locators & Interactions

- Locators: ID, NAME, XPATH
- Explicit waits
- Alerts, tables, iframes

MODULE 2

POM, DDT & PyTest

- Page Object Model
- CSV data-driven tests
- PyTest + HTML report

MODULE 3

BDD with Behave

- Gherkin scenarios
- Scenario Outline data
- Behave + Page Objects

Student Name	Sheefa Hussain
Batch	Computer Engineering and Information Technology
Date	25/09/2026
GitHub Repository	https://github.com/Sheefhuss/Selenium_Automation

Module 1: Core Fundamentals, Locators & Advanced Interactions

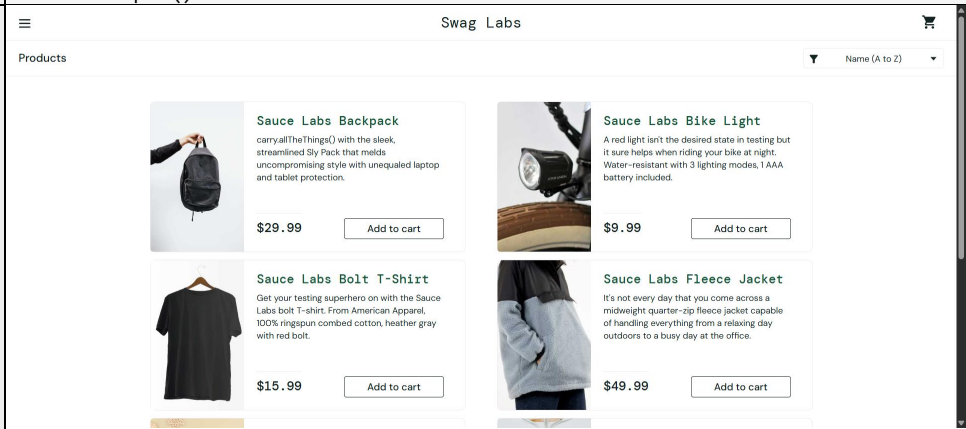
This module covers foundational Selenium automation concepts, moving from basic element locators and explicit waits to advanced browser interactions like alerts, web tables, and frames.

Video Demonstration: <https://drive.google.com/file/d/1Hc-3GKYIXKfSCKTEQqbLd7NVYQbQJt2B/view?usp=drivesdk>

Module folder: https://github.com/Sheefhuss/Selenium_Automation/tree/main/Modules/module1

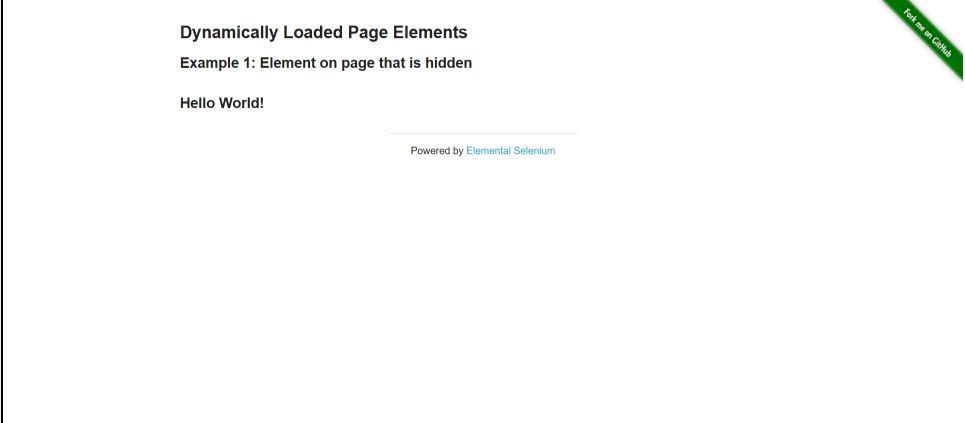
Experiment 1: The Multi-Locator Challenge

Problem Statement	A login page must be automated using more than one locator strategy for different elements, then the outcome of a successful login must be verified.
Objective	Interact with the username field using By.ID, the password field using By.NAME, and the login button using By.XPATH, then assert that the final URL contains /inventory.html.
Tools / Software / Concepts	• Python • Selenium WebDriver • By.ID / By.NAME / By.XPATH locator strategies • URL assertion
Implementation Steps	• Launch Chrome and navigate to https://www.saucedemo.com/. • Locate the username field using By.ID ('user-name') and enter 'standard_user'. • Locate the password field using By.NAME ('password') and enter 'secret_sauce'. • Locate the login button using By.XPATH and click it. • Assert that driver.current_url contains '/inventory.html', then save a screenshot.
Source Code	View on GitHub: Modules/module1/code_session/assignment1.py <pre>from selenium import webdriver from selenium.webdriver.common.by import By import time driver = webdriver.Chrome() driver.maximize_window() try: driver.get("https://www.saucedemo.com/") # Locate the username field by its id attribute username_field = driver.find_element(By.ID, "user-name") username_field.send_keys("standard_user") # Locate the password field by its name attribute password_field = driver.find_element(By.NAME, "password") password_field.send_keys("secret_sauce") # Locate the login button by an XPath expression login_button = driver.find_element(By.XPATH, "//input[@id='login-button']") login_button.click() time.sleep(1) current_url = driver.current_url assert "/inventory.html" in current_url, f"Login failed, landed on: {current_url}" print("PASS: Login successful, redirected to:", current_url)</pre>

	<pre>driver.save_screenshot("assignment1.png") finally: driver.quit()</pre>
Output (Screenshots)	 The screenshot shows the Sauce Labs inventory page. It features a header with the Sauce Labs logo and a shopping cart icon. Below the header, there's a 'Products' section with a dropdown menu set to 'Name (A to Z)'. Four product cards are displayed in a 2x2 grid. Each card includes a product image, name, description, price, and an 'Add to cart' button. The products are: Sauce Labs Backpack (\$29.99), Sauce Labs Bike Light (\$9.99), Sauce Labs Bolt T-Shirt (\$15.99), and Sauce Labs Fleece Jacket (\$49.99). <i>Fig 1.1 — Successful login redirect to the inventory page (saucedemo.com).</i>
Result / Observation / Conclusion	The objective was met: all three locator strategies (ID, NAME, XPATH) located their elements correctly, and the URL assertion confirmed a successful redirect to /inventory.html.

Experiment 2: Synchronization & Explicit Waits

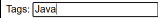
Problem Statement	Dynamic content on a page may not be available the instant a script tries to interact with it, and fixed delays (time.sleep()) are unreliable and slow.
Objective	Handle dynamic content correctly using WebDriverWait combined with expected_conditions instead of time.sleep().
Tools / Software / Concepts	- Python - Selenium WebDriverWait - expected_conditions (EC) - the-internet.herokuapp.com/dynamic_loading
Implementation Steps	- Navigate to the-internet.herokuapp.com/dynamic_loading/1. - Wait for the Start button to be clickable using EC.element_to_be_clickable, then click it. - Wait for the revealed text element to become visible using EC.visibility_of_element_located. - Print the revealed text and save a screenshot.
Source Code	View on GitHub: Modules/module1/code_session/assignment2.py <pre>Modules/module1/code_session/assignment2.py from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support import expected_conditions as EC driver = webdriver.Chrome() driver.maximize_window() try: driver.get("https://the-internet.herokuapp.com/dynamic_loading/1") start_button = WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.CSS_SELECTOR, "#start button"))) start_button.click() # Wait until the hidden text box is visible and loaded revealed_text = WebDriverWait(driver, 10).until(</pre>

	<pre> EC.visibility_of_element_located((By.ID, "finish"))) print("Revealed text:", revealed_text.text) driver.save_screenshot("assignment2.png") finally: driver.quit() </pre>
Output (Screenshots)	 Fig 1.2 — Hidden text revealed after the explicit wait condition is satisfied.
Result / Observation / Conclusion	No time.sleep() was used. Both WebDriverWait conditions (element_to_be_clickable, visibility_of_element_located) worked reliably, confirming synchronization was handled correctly instead of using a fixed delay.

Experiment 3: Checkbox Toggling & Autocomplete Dropdown Selection

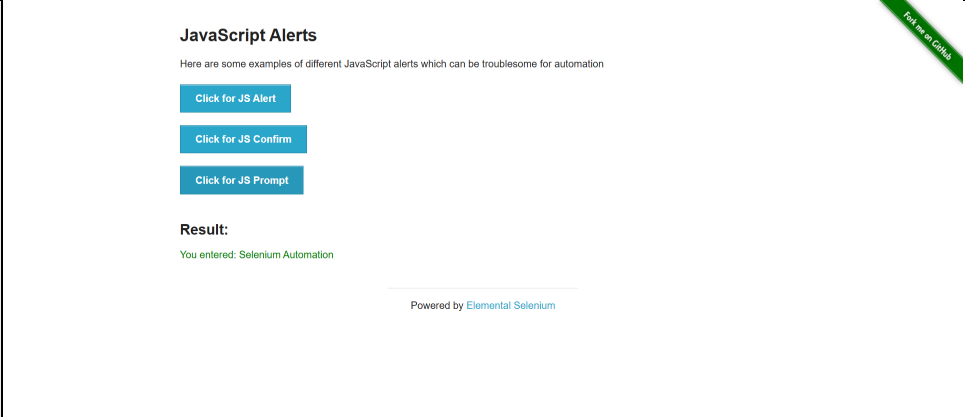
Problem Statement	A set of checkboxes needs their state read and toggled, and a separate autocomplete widget needs a specific suggestion selected from a dynamically-appearing list — an internal practice exercise combining element lists with explicit waits.
Objective	Iterate through all checkboxes on a page and ensure each is selected, then type into an autocomplete field and select the exact suggestion 'Java' once the suggestion list becomes visible.
Tools / Software / Concepts	• Python • Selenium find_elements • WebDriverWait / expected_conditions • By.CSS_SELECTOR
Implementation Steps	• Part A: Open the-internet.herokuapp.com/checkboxes, get all checkbox elements, click any that are not already selected, and print each one's selected state. • Save a screenshot of the checkbox state (screenshot_checkbox.png). • Part B: Open jqueryui.com's autocomplete demo, wait for the search box, and type 'Ja'. • Wait for the suggestion list to become visible, loop through the suggestions, and click the one matching 'Java'. • Save a screenshot of the completed selection (assignment3.png).
Source Code	View on GitHub: Modules/module1/code_session/assignment3.py <pre> Modules/module1/code_session/assignment3.py from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support import expected_conditions as EC driver = webdriver.Chrome() </pre>

	<pre>driver.maximize_window() try: # --- Part A: Checkboxes --- driver.get("https://the-internet.herokuapp.com/checkboxes") checkboxes = driver.find_elements(By.CSS_SELECTOR, "#checkboxes input[type='checkbox']") for i, box in enumerate(checkboxes): if not box.is_selected(): box.click() print(f"Checkbox {i + 1} selected state: {box.is_selected()}") driver.save_screenshot("screenshot_checkbox.png") # --- Part B: Autocomplete dropdown --- driver.get("https://jqueryui.com/resources/demos/autocomplete/default.html") wait = WebDriverWait(driver, 10) # Directly wait for the search box to be present search_box = wait.until(EC.presence_of_element_located((By.ID, "tags"))) search_box.send_keys("Ja") # Wait for suggestions list to appear suggestions = wait.until(EC.visibility_of_all_elements_located((By.CSS_SELECTOR, "ul.ui-autocomplete li"))) target = "Java" for suggestion in suggestions: if suggestion.text.strip() == target: suggestion.click() print(f"Selected suggestion: {target}") break driver.save_screenshot("assignment3.png") finally: driver.quit()</pre>
Output (Screenshots)	Checkboxes ☒ checkbox 1 ☒ checkbox 2 Powered by Elemental Selenium Fig 1.3a — All checkboxes toggled to the selected state.

	 Tags: Java <i>Fig 1.3b — 'Java' suggestion selected from the autocomplete dropdown.</i>
Result / Observation / Conclusion	Both parts completed successfully: all checkboxes reached the selected state, and the autocomplete suggestion matching 'Java' was correctly identified and clicked after the explicit wait for suggestion visibility.

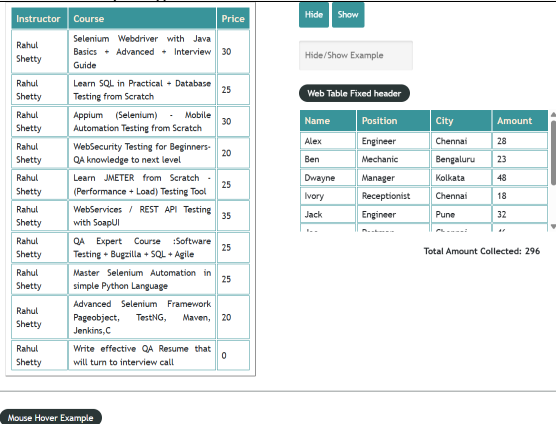
Experiment 4: JavaScript Alerts and Confirms

Problem Statement	Native JavaScript Alert, Confirm, and Prompt dialogs sit outside the normal DOM and cannot be interacted with using standard element locators.
Objective	Trigger, accept, dismiss, and enter text into JavaScript Alert, Confirm, and Prompt boxes using <code>driver.switch_to.alert</code> .
Tools / Software / Concepts	• Selenium Alert API • <code>driver.switch_to.alert</code> • <code>WebDriverWait + EC.alert_is_present()</code>
Implementation Steps	• Navigate to <code>the-internet.herokuapp.com/javascript_alerts</code>. • Click 'Click for JS Alert', wait for the alert, print its text, and accept it. • Click 'Click for JS Confirm', wait for the alert, print its text, and dismiss it. • Click 'Click for JS Prompt', send text 'Selenium Automation', and accept it. • Save a screenshot after all three dialogs are handled.
Source Code	View on GitHub: Modules/module1/code_session/assignment4.py <pre> Modules/module1/code_session/assignment4.py from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support import expected_conditions as EC driver = webdriver.Chrome() driver.maximize_window() try: driver.get("https://the-internet.herokuapp.com/javascript_alerts") # 1. Accept JS Alert driver.find_element(By.XPATH, "//button[text()='Click for JS Alert']").click() alert = WebDriverWait(driver, 5).until(EC.alert_is_present()) print("Alert text:", alert.text) alert.accept() # 2. Dismiss JS Confirm Box driver.find_element(By.XPATH, "//button[text()='Click for JS Confirm']").click() confirm = WebDriverWait(driver, 5).until(EC.alert_is_present()) print("Confirm text:", confirm.text) confirm.dismiss() </pre>

	<pre># 3. Input Text into JS Prompt Box & Accept driver.find_element(By.XPATH, "//button[text()='Click for JS Prompt']").click() prompt = WebDriverWait(driver, 5).until(EC.alert_is_present()) prompt.send_keys("Selenium Automation") prompt.accept() # Save screenshot driver.save_screenshot("assignment4_screenshot_js_alerts.png") finally: driver.quit()</pre>
Output (Screenshots)	 <i>Fig 1.4 — Page state after the Alert, Confirm, and Prompt dialogs were handled.</i>
Result / Observation / Conclusion	All three dialog types were triggered and handled without exception: the Alert was accepted, the Confirm was dismissed, and text was successfully entered into the Prompt before accepting it.

Experiment 5: The HTML Web Table Extractor

Problem Statement	Specific values (such as a status or price) must be retrieved from a dynamic, multi-column HTML table without hardcoding row positions.
Objective	Iterate through the rows and columns of a course-listing web table, match the row named 'Master Selenium Automation in simple Python Language', and extract its price.
Tools / Software / Concepts	• Selenium find_element(s) • By.XPATH / By.TAG_NAME • Row/column iteration
Implementation Steps	• Navigate to rahulshettyacademy.com/AutomationPractice and locate the courses table. • Scroll the table into view. • Loop through every <tr>, and within each row every <td>. • Compare the course-name cell against the target course name. • On a match, print the course name and extracted price, then save a screenshot.
Source Code	View on GitHub: Modules/module1/code_session/assignment5.py Modules/module1/code_session/assignment5.py <pre>from selenium import webdriver from selenium.webdriver.common.by import By import time driver = webdriver.Chrome() driver.maximize_window() try: driver.get("https://rahulshettyacademy.com/AutomationPractice/") table = driver.find_element(By.XPATH, "//table[@name='courses']")</pre>

	<pre> driver.execute_script("arguments[0].scrollIntoView(true);", table) time.sleep(5) target_course = "Master Selenium Automation in simple Python Language" rows = table.find_elements(By.TAG_NAME, "tr") for row in rows: cols = row.find_elements(By.TAG_NAME, "td") if cols: course_name = cols[1].text.strip() price = cols[2].text.strip() if course_name == target_course: print(f"Matched Course: '{course_name}'") print(f"Extracted Price: {price}") break driver.save_screenshot("assignment5_web_tables.png") finally: driver.quit() </pre>
Output (Screenshots)	 The screenshot displays two web tables. The first table, titled 'Courses', lists various courses with their respective instructors and prices. The second table, titled 'Employees', lists employee names, positions, cities, and amounts. A 'Web Table Fixed header' button is visible above the second table. Fig 1.5 — Course table scrolled into view before row-matching.
Result / Observation / Conclusion	The target row was correctly matched by course name, and its price cell was extracted and printed, confirming reliable row/column iteration over a dynamic table.

Experiment 6: Windows, Tabs, and Iframes

Problem Statement	A single script may need to interact with content embedded in an iframe and also with content that opens in a separate browser tab.
Objective	Switch context correctly between an embedded iframe and a newly opened browser tab using <code>window_handles</code> and <code>switch_to.frame()</code> , confirming each context switch with a JS alert.
Tools / Software / Concepts	• <code>driver.window_handles</code> • <code>driver.switch_to.window()</code> • <code>driver.switch_to.frame()</code> • <code>driver.switch_to.default_content()</code>
Implementation Steps	• Open rahulshettyacademy.com/AutomationPractice and record the main window handle. • Scroll to and switch into the courses iframe using <code>switch_to.frame()</code>, confirming with an injected alert. • Switch back to the main page with <code>switch_to.default_content()</code>.

	● Click the 'Open Tab' button, wait for a second window handle to appear, and switch to it. ● Read the new tab's title, save a screenshot, close the tab, and switch back to the original window handle.
Source Code	View on GitHub: Modules/module1/code_session/assignment6.py Modules/module1/code_session/assignment6.py <pre> from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support import expected_conditions as EC import time def show_and_accept_alert(driver, message): """Triggers an alert safely, pauses so you can read it, then accepts it.""" driver.execute_script("alert(arguments[0]);", message) time.sleep(1) driver.switch_to.alert.accept() time.sleep(1) driver = webdriver.Chrome() driver.maximize_window() try: driver.get("https://rahulshettyacademy.com/AutomationPractice/") main_window_handle = driver.current_window_handle show_and_accept_alert(driver, "TASK 1 START: Main page loaded. Next, scrolling down to the IFrame.") # HANDLE IFRAME iframe_element = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.ID, "courses-iframe"))) driver.execute_script("arguments[0].scrollIntoView(true);", iframe_element) time.sleep(3) driver.switch_to.frame(iframe_element) show_and_accept_alert(driver, "INSIDE IFRAME: Driver focus is now inside the embedded frame!") driver.switch_to.default_content() show_and_accept_alert(driver, "BACK TO MAIN PAGE: Exited iframe. Scroll back to top for tab test.") driver.execute_script("window.scrollTo(0, 0);") time.sleep(3) show_and_accept_alert(driver, "TASK 2 START: Clicking the 'Open Tab' button now.") driver.find_element(By.ID, "opentab").click() WebDriverWait(driver, 5).until(lambda d: len(d.window_handles) > 1) all_handles = driver.window_handles for handle in all_handles: if handle != main_window_handle: driver.switch_to.window(handle) break time.sleep(3) new_tab_title = driver.title show_and_accept_alert(driver, f"NEW TAB DETECTED! Context switched. Title: '{new_tab_title}'") time.sleep(3) # Take screenshot driver.save_screenshot("assignment6_screenshot_windows_iframes.png") </pre>

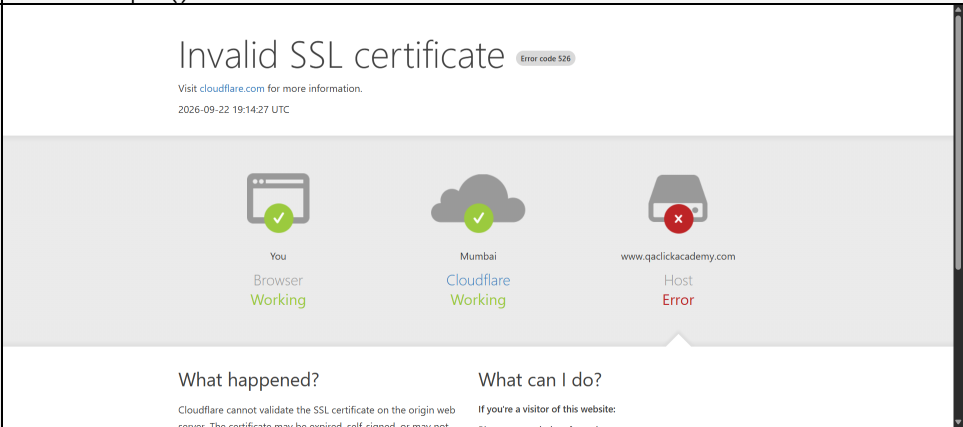
	<pre>driver.close() # RETURN TO ORIGINAL TAB driver.switch_to.window(main_window_handle) show_and_accept_alert(driver, "EXPERIMENT 6 COMPLETE: Successfully returned to original main layout!") finally: driver.quit()</pre>			
Output (Screenshots)	 Invalid SSL certificate Error code 526 Visit cloudflare.com for more information. 2026-09-22 19:14:27 UTC <table><tr><td> You Browser Working</td><td> Mumbai Cloudflare Working</td><td> www.qaclickacademy.com Host Error</td></tr></table> What happened? Cloudflare cannot validate the SSL certificate on the origin web cause. The certificate may be expired, self signed, or may not What can I do? If you're a visitor of this website:	 You Browser Working	 Mumbai Cloudflare Working	 www.qaclickacademy.com Host Error
 You Browser Working	 Mumbai Cloudflare Working	 www.qaclickacademy.com Host Error		
Result / Observation / Conclusion	Context switching worked in both directions: iframe -> default content, and main window -> new tab -> back to the main window, with the new tab's title correctly retrieved before it was closed.			

Fig 1.6 — New tab detected and its title read after the context switch.

Module 2: Page Object Model, Data-Driven Testing & PyTest

This module advances automation architecture by implementing the Page Object Model (POM), data-driven testing (DDT) using external files, and scalable test execution using PyTest with HTML reporting.

Video Demonstration: <https://drive.google.com/file/d/1wvSgMq75kBy-12QaSVnMi0MIKBuMpG9j/view?usp=drivesdk>

Module folder: https://github.com/Sheefhuss/Selenium_Automation/tree/main/Modules/module2

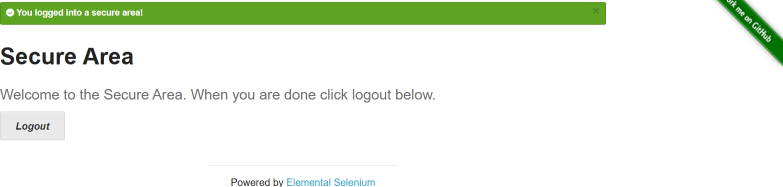
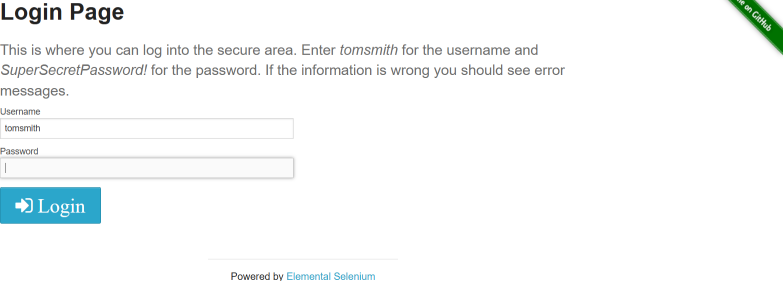
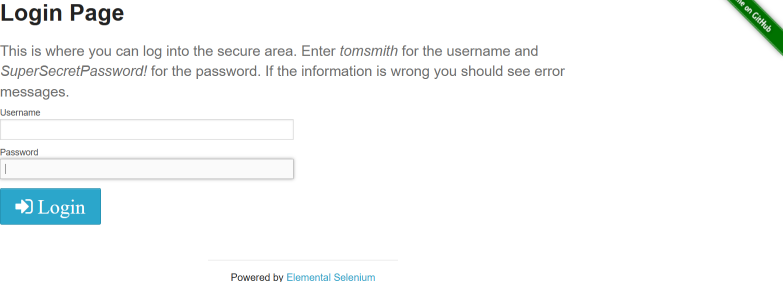
Experiment 7: Page Object Model (POM) Restructure

Problem Statement	Working scripts that mix locators, actions, and assertions in one file are hard to maintain and reuse.
Objective	Restructure a login script into a Page Object Model: a LoginPage class holding only locators and UI actions, with the test itself performing only the assertion.
Tools / Software / Concepts	• Python • Selenium WebDriver • Page Object Model (POM) design pattern
Implementation Steps	• Create login_page.py defining a LoginPage class with username/password/login_button locators. • Add enter_username(), enter_password(), and click_login() methods to LoginPage. • Create test_login.py, which imports LoginPage, drives the three actions, and performs the assertion. • Assert that '/secure' is present in driver.current_url after login.
Source Code	View on GitHub: Modules/module2/assignment7 Modules/module2/assignment7/login_page.py <pre>from selenium.webdriver.common.by import By class LoginPage: def __init__(self, driver): self.driver = driver username = (By.ID, "username") password = (By.ID, "password") login_button = (By.CSS_SELECTOR, "button[type='submit']") def enter_username(self, username): self.driver.find_element(*self.username).send_keys(username) """self.username is: By.ID, "username" The * unpacks the tuple.""" def enter_password(self, password): self.driver.find_element(*self.password).send_keys(password) def click_login(self): self.driver.find_element(*self.login_button).click()</pre> Modules/module2/assignment7/test_login.py <pre>from selenium import webdriver from login_page import LoginPage def test_login(): driver = webdriver.Chrome() driver.get("https://the-internet.herokuapp.com/login")</pre>

	<pre>login = LoginPage(driver)#login is an object of LoginPage login.enter_username("tomsmith") login.enter_password("SuperSecretPassword!") login.click_login() assert "/secure" in driver.current_url driver.quit() test_login()</pre>
Output (Screenshots)	
Result / Observation / Conclusion	The locators and UI actions live entirely in LoginPage, and test_login.py contains only the assertion, meeting the POM separation objective. Note: test_login() is called directly as a plain function rather than run through pytest — Experiment 9 introduces pytest-based execution.

Experiment 8: Data-Driven Automation (DDT)

Problem Statement	Testing multiple username/password combinations by hardcoding each case is repetitive and does not scale.
Objective	Read login test cases from an external CSV file using csv.DictReader and loop through them, asserting the expected outcome (success or invalid) for each row.
Tools / Software / Concepts	• csv.DictReader • Selenium WebDriver • login_data.csv (4 rows)
Implementation Steps	• Read modules/module2/login_data.csv, with columns username, password, expected. • For each row, navigate to the login page, enter the given username/password, and submit. • Save a screenshot named after the row's username and expected result. • Assert '/secure' in the URL when expected == 'success', otherwise assert '/login' in the URL.
Source Code	View on GitHub: Modules/module2/assignment8.py Modules/module2/assignment8.py <pre>import csv from selenium import webdriver from selenium.webdriver.common.by import By driver = webdriver.Chrome() driver.maximize_window() with open("modules/module2/login_data.csv", "r") as file: data = csv.DictReader(file) for row in data: driver.get("https://the-internet.herokuapp.com/login") driver.find_element(By.ID, "username").send_keys(row["username"]) driver.find_element(By.ID, "password").send_keys(row["password"]) driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click() driver.save_screenshot("Assignment_8_row_" + row["username"] + "_" + row["expected"] + ".png") if row["expected"] == "success": assert "/secure" in driver.current_url print(row["username"], "LOGIN SUCCESS") else: assert "/login" in driver.current_url</pre>

	<pre>print(row["username"], "LOGIN FAILED - VALIDATION CORRECT") driver.quit()</pre>
Output (Screenshots)	 <i>Fig 2.1 — Row 1 (tomsmith / correct password): successful login.</i>  <i>Fig 2.2 — Row 3 (tomsmith / wrong password): validation error shown.</i>  <i>Fig 2.3 — Row 2 or 4 (wronguser): validation error shown.</i>
Result / Observation / Conclusion	All 4 CSV rows executed and asserted the correct outcome. Observation: the screenshot filename is built from username + expected result only, so the two 'wronguser' rows (rows 2 and 4) both save to the same filename and the second run overwrites the first — only 3 distinct screenshot files exist for 4 test rows. This does not affect the assertions, only the retained screenshot evidence.

Experiment 9: PyTest Integration with HTML Reporting

Problem Statement	Running scripts directly and manually inspecting results does not scale, and there is no persistent, shareable record of results across multiple data-driven cases.
Objective	Convert the DDT login test to run via PyTest, using a fixture for browser setup/teardown and @pytest.mark.parametrize to drive it from the same CSV data, then generate an HTML report.
Tools / Software / Concepts	• pytest • pytest.fixture • pytest.mark.parametrize • pytest-html
Implementation Steps	• Define a driver fixture that opens Chrome, navigates to the login URL, yields the driver, then quits it. • Read the same login_data.csv used in Assignment 8 via read_test_data(). • Parametrize test_login(driver, data) over every CSV row. • Assert '/secure' for success rows and '/login' for invalid rows. • Run: pytest test_assignment9.py --html=report.html --self-contained-html
Source Code	View on GitHub: Modules/module2/test_assignment9.py Modules/module2/test_assignment9.py <pre>import csv import pytest from selenium import webdriver from selenium.webdriver.common.by import By URL = "https://the-internet.herokuapp.com/login" @pytest.fixture def driver(): driver = webdriver.Chrome() driver.maximize_window() driver.get(URL) yield driver driver.quit() def read_test_data(): with open("modules/module2/login_data.csv", "r") as file: return list(csv.DictReader(file)) @pytest.mark.parametrize("data", read_test_data()) def test_login(driver, data): driver.find_element(By.ID, "username").send_keys(data["username"]) driver.find_element(By.ID, "password").send_keys(data["password"]) driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click() if data["expected"] == "success": assert "/secure" in driver.current_url else: assert "/login" in driver.current_url</pre>

Output (Screenshots)	report.html Report generated on 24-Sep-2026 at 01:32:03 by pytest.html v4.2.0 Environment Python 3.13.11 Platform Windows-11-10.0.26200-SP0 Packages • pytest: 9.1.1 • pluggy: 1.6.0 Plugins • html: 4.2.0 • metadata: 3.1.1 Summary 4 tests took 00:00:35 (X)check the boxes to filter the results ☐ 0 Failed ☒ 4 Passed ☐ 0 Skipped ☐ 0 Expected failures ☐ 0 Unexpected passes ☐ 0 Errors ☐ 0 Reruns ☐ 0 Retried Show all details / Hide all details <table><tr><th>Result</th><th>Test</th><th>Duration</th><th>Links</th></tr><tr><td>Passed</td><td>modules/module2/test_assignment0 py: test_login(data0)</td><td>00:00:10</td><td></td></tr><tr><td>Passed</td><td>modules/module2/test_assignment0 py: test_login(data1)</td><td>00:00:08</td><td></td></tr><tr><td>Passed</td><td>modules/module2/test_assignment0 py: test_login(data2)</td><td>00:00:08</td><td></td></tr><tr><td>Passed</td><td>modules/module2/test_assignment0 py: test_login(data3)</td><td>00:00:08</td><td></td></tr></table> Fig 2.4 — PyTest run / generated HTML report for the parametrized login tests.	Result	Test	Duration	Links	Passed	modules/module2/test_assignment0 py: test_login(data0)	00:00:10		Passed	modules/module2/test_assignment0 py: test_login(data1)	00:00:08		Passed	modules/module2/test_assignment0 py: test_login(data2)	00:00:08		Passed	modules/module2/test_assignment0 py: test_login(data3)	00:00:08	
Result	Test	Duration	Links																		
Passed	modules/module2/test_assignment0 py: test_login(data0)	00:00:10																			
Passed	modules/module2/test_assignment0 py: test_login(data1)	00:00:08																			
Passed	modules/module2/test_assignment0 py: test_login(data2)	00:00:08																			
Passed	modules/module2/test_assignment0 py: test_login(data3)	00:00:08																			
Result / Observation / Conclusion	All parametrized cases ran under pytest and produced a single HTML report (report.html) alongside the console output. Observation: the current script does not include a screenshot-on-failure hook (e.g. a <code>pytest_runtest_makereport</code> hook) as described in the module brief — the HTML report itself is generated, but failure screenshots are not yet automatically embedded in it.																				

Fig 2.4 — PyTest run / generated HTML report for the parametrized login tests.

Module 3: Behavior-Driven Development (BDD) with Behave

This module focuses on implementing Behavior-Driven Development (BDD) using the Python Behave framework, integrating Gherkin language scenarios, data-driven structures, and combining Behave with the Selenium Page Object Model.

Video Demonstration: <https://drive.google.com/file/d/1WiptF2DALU4c3YVSAf-zMXh4zJPH6Fhp/view?usp=drivesdk>

Module folder: https://github.com/Sheefhuss/Selenium_Automation/tree/main/Modules/module3

Experiment 1: Implementation of Selenium Python and Behave BDD

Problem Statement	End-to-end scenarios need to be described and executed in a business-readable format rather than as raw Python test functions.
Objective	Set up the Python BDD framework and run an end-to-end login scenario written in Gherkin, with Selenium step definitions.
Tools / Software / Concepts	• Behave (BDD framework for Python) • Gherkin syntax (Given / When / Then) • Selenium WebDriver • PyCharm / VS Code
Implementation Steps	• Write features/login.feature describing a 'Successful login' scenario in Gherkin. • Implement features/steps/login_steps.py: open the login page, enter username and password, click login. • In the Then step, assert '/secure' is in the URL and quit the driver. • Run the feature from the assignment1 folder with the behave command.
Source Code	View on GitHub: Modules/module3/assignment1 assignment1/features/login.feature Feature: Login functionality <pre>Scenario: Successful login Given I open the login page When I enter username "tomsmith" And I enter password "SuperSecretPassword!" And I click the login button Then I should be successfully logged in</pre> assignment1/features/steps/login_steps.py <pre>from behave import given, when, then from selenium import webdriver from selenium.webdriver.common.by import By @given("I open the login page") def open_login_page(context): context.driver = webdriver.Chrome() context.driver.get("https://the-internet.herokuapp.com/login") @when('I enter username "tomsmith"') def enter_username(context): context.driver.find_element(By.ID, "username").send_keys("tomsmith") @when('I enter password "SuperSecretPassword!"') def enter_password(context): context.driver.find_element(By.ID, "password").send_keys("SuperSecretPassword!")</pre>

	<pre> @when("I click the login button") def click_login(context): context.driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click() @then("I should be successfully logged in") def verify_login(context): assert "/secure" in context.driver.current_url context.driver.quit() </pre>
Output (Screenshots)	<pre> (venv) PS C:\Users\sheef\project0\modules\module3\assignment1> behave USING RUNNER: behave.runner:Runner Feature: Login functionality # features/login.feature:1 Then I should be successfully logged in # features/steps/login_steps.py:27 2.882s Then I should be successfully logged in # features/steps/login_steps.py:27 1 feature passed, 0 failed, 0 skipped 1 scenario passed, 0 failed, 0 skipped 5 steps passed, 0 failed, 0 skipped Fig 3.1 — behave run: 1 feature, 1 scenario, 5 steps passed. </pre>
Result / Observation / Conclusion	The Gherkin scenario ran end to end: 1 feature, 1 scenario and all 5 steps passed with 0 failures, confirming the Behave + Selenium setup works.

Experiment 2: Test Data-Driven Automation in Python Behave Framework

Problem Statement	A single Gherkin scenario needs to be executed against multiple sets of input data without duplicating the scenario text.
Objective	Adapt the BDD framework to handle multiple test cases using a Scenario Outline with an Examples table.
Tools / Software / Concepts	- Behave - Gherkin Scenario Outline / Examples table - Parameterised step definitions - Python
Implementation Steps	- Convert the scenario into a Scenario Outline with <username>, <password> and expected-result placeholders. - Add an Examples table with 4 rows of valid and invalid credentials. - Use parameterised steps ({username}, {password}, {result}) in login_steps.py. - In the Then step, assert '/secure' for success and '/login' for failure, then run behave.
Source Code	View on GitHub: Modules/module3/assignment2/assignment2/features/login.feature <pre> Feature: Login functionality Scenario Outline: Login with different test data Given I open the login page When I enter username "<username>" And I enter password "<password>" And I click the login button Then I should get "<result>" Examples: username password result tomsmith SuperSecretPassword success wronguser SuperSecretPassword! failure tomsmith wrongpassword failure wronguser wrongpassword failure </pre> assignment2/features/steps/login_steps.py <pre> from behave import given, when, then from selenium import webdriver from selenium.webdriver.common.by import By </pre>

	<pre> @given("I open the login page") def open_login_page(context): context.driver = webdriver.Chrome() context.driver.get("https://the-internet.herokuapp.com/login") @when('I enter username "{username}"') def enter_username(context, username): context.driver.find_element(By.ID, "username").send_keys(username) @when('I enter password "{password}"') def enter_password(context, password): context.driver.find_element(By.ID, "password").send_keys(password) @when("I click the login button") def click_login(context): context.driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click() @then('I should get "{result}"') def verify_login(context, result): if result == "success": assert "/secure" in context.driver.current_url else: assert "/login" in context.driver.current_url context.driver.quit() </pre>
Output (Screenshots)	<pre> (venv) PS C:\Users\sheef\project0\modules\module3\assignment2> behave USING RUNNER: behave.runner:Runner Feature: Login functionality # features/login.feature:1 Then I should get "success" # features/steps/login_steps.py:27 2.843s Then I should get "success" # features/steps/login_steps.py:27 Then I should get "failure" # features/steps/login_steps.py:27 2.847s Then I should get "failure" # features/steps/login_steps.py:27 Then I should get "failure" # features/steps/login_steps.py:27 2.225s Then I should get "failure" # features/steps/login_steps.py:27 Then I should get "failure" # features/steps/login_steps.py:27 2.221s Then I should get "failure" # features/steps/login_steps.py:27 1 feature passed, 0 failed, 0 skipped 4 scenarios passed, 0 failed, 0 skipped 20 steps passed, 0 failed, 0 skipped Took 0min 30.190s - </pre> Fig 3.2 — behave run: 4 scenarios (one per Examples row), 20 steps passed.
Result / Observation / Conclusion	Each Examples row ran as its own scenario: 4 scenarios and 20 steps passed, 0 failed (about 30 s). Observation: in the pushed feature file the first row's password is 'SuperSecretPassword' without the trailing '!' while the expected result is 'success' — the passing run above implies the correct password was used, so please re-check that row of the committed file.

Experiment 3: Selenium Page Object Model in the Python Behave Framework

Problem Statement	Step definitions written directly against locators are hard to maintain and do not reuse the POM structure built in Module 2; environment settings are also hard-coded.
Objective	Combine Behave with the Page Object Model, and drive the base URL and browser from a config.ini file, with browser setup/teardown handled by Behave hooks.
Tools / Software / Concepts	- Behave - Gherkin - Page Object Model (LoginPage) - configparser + config.ini - environment.py hooks

Implementation Steps	• Create pages/login_page.py holding the locators and enter_username / enter_password / click_login methods. • Store environment, base_url and browser in config.ini. • In environment.py, read config.ini in before_all, start Chrome in before_scenario and quit it in after_scenario. • Write steps that call LoginPage methods only, and verify the login with WebDriverWait + EC.url_contains('/secure'). • Run behave from the assignment3 folder.
Source Code	View on GitHub: Modules/module3/assignment3 assignment3/features/login.feature Feature: Login functionality Scenario: Successful login Given I open the login page When I enter username "tomsmith" And I enter password "SuperSecretPassword!" And I click the login button Then I should be successfully logged in assignment3/features/pages/login_page.py <pre> from selenium.webdriver.common.by import By class LoginPage: def __init__(self, driver): self.driver = driver #Locators self.username_field = (By.ID, "username") self.password_field = (By.ID, "password") self.login_button = (By.CSS_SELECTOR, "button[type='submit']") def enter_username(self, username): self.driver.find_element(*self.username_field).send_keys(username) def enter_password(self, password): self.driver.find_element(*self.password_field).send_keys(password) def click_login(self): self.driver.find_element(*self.login_button).click() </pre> assignment3/features/steps/login_steps.py <pre> from behave import given, when, then from selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support import expected_conditions as EC from pages.login_page import LoginPage @given("I open the login page") def open_login_page(context): # Uses the dynamic URL loaded from config.ini via environment.py context.driver.get(context.base_url) context.login = LoginPage(context.driver) @when('I enter username "{username}"') def enter_username(context, username): context.login.enter_username(username) @when('I enter password "{password}"') def enter_password(context, password): context.login.enter_password(password) @when("I click the login button") def click_login(context): context.login.click_login() @then("I should be successfully logged in") def verify_login(context): </pre>

	<pre> WebDriverWait(context.driver, 10).until(EC.url_contains("/secure")) assert "/secure" in context.driver.current_url assignment3/features/environment.py import configparser from selenium import webdriver def before_all(context): config = configparser.ConfigParser() config.read('config.ini') env = config.get('DEFAULT', 'environment') context.base_url = config.get(env, 'base_url') context.browser_type = config.get(env, 'browser') def before_scenario(context, scenario): if context.browser_type == 'chrome': context.driver = webdriver.Chrome() context.driver.maximize_window() def after_scenario(context, scenario): if hasattr(context, 'driver'): context.driver.quit() assignment3/config.ini [DEFAULT] environment = staging timeout = 10 [staging] base_url = https://the-internet.herokuapp.com/login browser = chrome </pre>
Output (Screenshots)	<pre> (env) PS C:\Users\sheef\project0\modules\module3\assignment3> behave USING RUNNER: behave.runner:Runner Feature: Login functionality # features/login.feature:1 Then I should be successfully logged in # features/steps/login_steps.py:24 0.656s Then I should be successfully logged in # features/steps/login_steps.py:24 1 feature passed, 0 failed, 0 skipped 1 scenario passed, 0 failed, 0 skipped 5 steps passed, 0 failed, 0 skipped Took 0min 4.946s </pre> <i>Fig 3.3 — behave run using Page Objects and config.ini: 1 scenario, 5 steps passed (about 5 s).</i>
Result / Observation / Conclusion	The scenario passed (5 steps, 0 failed). Locators live only in LoginPage, the URL and browser come from config.ini, and driver setup/teardown is centralised in environment.py, so step definitions stay short and reusable.

Overall Summary

All three modules are implemented, pushed to the GitHub repository and documented above with real source code and screenshots from actual runs. Module 1 covers locators, explicit waits, alerts, tables and frames;

Module 2 restructures the work into Page Objects, drives it from CSV data and runs it under PyTest with an HTML report;

Module 3 expresses the same login flow as Gherkin scenarios in Behave, then adds data-driven examples and Page Objects. Two follow-ups are noted in the experiment rows: Experiment 9 has no screenshot-on-failure hook, and the Module 3 Assignment 2 data row should be re-checked.

Full repository: https://github.com/Sheefhuss/Selenium_Automation